CAREER ROADMAP

Java / Python Backend Engineering, Data Engineering, DevOps and AI

One master document with three coordinated roadmaps

This guide consolidates the eight uploaded plans. Repeated Java-to-AI plans are combined; the two low-latency roadmaps become an optional specialist branch; data engineering and DevOps are made into a distinct track. The goal is to land a job first, then broaden depth without trying to learn every stack at once.

ROADMAP 1	Job-first Java/Python backend and senior software-engineering fundamentals. Start now; this is the foundation for everything else.
ROADMAP 2	Data engineering plus DevOps/platform. Build on backend, SQL, Git, containers and one cloud; turn the same project into a reliable data pipeline.
ROADMAP 3	AI engineering plus data-science/ML foundations. Start with AI-enabled backend work; add deeper statistics and model training as a deliberate specialization.

The most important rule

Do not treat these as three simultaneous full-time curricula. Prioritize Roadmap 1 for job readiness, apply from week 1, and grow Roadmaps 2 and 3 by upgrading one portfolio project. Courses support the project; the project does not wait for every course.

Your available learning resources

Use Coursera Plus, Pluralsight, Udemy titles available through your BCCLS library card, and free resources from MIT OpenCourseWare, Khan Academy, YouTube, official documentation and DeepLearning.AI. Verify exact course access before scheduling it.

Default strategy: keep Java as the primary enterprise backend language; use Python for data processing, automation, AI services and ML. Learn one cloud deeply, not AWS and GCP together.

Execution plan: milestones, job search and weekly rhythm

Use this as a flexible pacing guide, not a promise about hiring dates. At about 12 focused hours per week, the job-ready core comes first; cloud and AI deepen the same portfolio project afterward.

Operating rule Apply to suitable backend roles now, keep DSA and interview practice running every week, and tie every course to a capstone commit, test, design note or deployment. Do not wait to finish Kafka, Kubernetes, cloud certification or AI before applying. If you get an interview early, temporarily switch to interview mode.

Roadmap checkpoints (about 32 weeks at 12 hours/week)

Phase	Approx. timing	Focus	Exit evidence and job-search target
1. Tools + first build	Weeks 1-4	Linux basics, Git/GitHub, Docker/Compose, CI, Spring Boot refresh and DSA. Start one capstone repo.	Clean clone runs locally; tests and README exist; one-page resume, LinkedIn/GitHub refreshed, start behavioral stories; aim for ~20 DSA problems. Begin applying now; target ~5 tailored applications/week by the end of this phase.
2. Java depth + data	Weeks 5-10	Concurrency and Java Memory Model; PostgreSQL, transactions, indexes, migrations, SQL plans and tests.	Working backend MVP; explain volatile/atomic/synchronized and happens-before; read EXPLAIN. Aim for ~60 DSA problems. Increase to ~10-15 targeted applications/week when materials are ready.
3. Design + events	Weeks 11-16	System design; Kafka; idempotency, outbox, retries/DLQ; developer-level Kubernetes.	One clear design document, six practiced designs, working event flow and local deployment; aim for ~100 DSA problems and one mock interview/week. Aim for ~15-25 targeted applications/week if quality and follow-up remain strong.
4. Cloud + operations	Weeks 17-24	Go deep on one cloud (default AWS here); IAM, networking, deploy, secrets, logs/metrics, backups, cost and Terraform basics.	Live or reproducibly deployed capstone, architecture diagram, runbook, cost guardrails; aim for ~130 DSA problems reviewed. Keep applications and mocks going.
5. AI or data specialization	Weeks 25-32	Choose AI backend/RAG or data engineering/platform first. For AI: Python/FastAPI, retrieval and evaluation. For data: modeling, pipelines, quality and orchestration.	One measurable specialization extension: RAG evaluation report OR a reliable data pipeline. Fine-tuning, multi-agent systems and advanced low-latency work remain optional.

Weekly operating rhythm (12 hours)

3 hours	DSA and coding interviews: solve in Java, revisit missed problems, explain complexity aloud.
2 hours	Job search: prioritize referrals/past colleagues and recruiters, then targeted applications; tailor resume bullets and track outcomes.
4 hours	Capstone and labs: implement, test, document, measure and commit. The repository grows through every phase.
2 hours	One course or documentation topic that unblocks the next project step. Do not run several parallel courses.
1 hour	Review notes, record a short architecture decision, update mistakes log and plan the next week.

DSA and interview loop

Practice method
Use NeetCode 150 in Java as a pattern checklist; Princeton Algorithms, Part I is optional theory support. Try a problem cold for ~25 minutes, take a hint, study the solution, then re-solve it about a week later. Keep a mistake log. Aim for roughly 4-6 thoughtful problems per week, not a rushed count.

Every week
One system-design sketch, one Java/concurrency or SQL debugging question, and one behavioral story. From the system-design phase, do one timed mock weekly. If an interview arrives early, pause new courses for up to two weeks and focus on that employer's likely coding, design and behavioral loop.

When overloaded: reduce extra courses, optional certifications, advanced ML depth and low-latency specialization first. Protect DSA review, concurrency fundamentals, system-design drills, targeted job search and a weekly capstone commit. If a McDaniel crunch week hits, use a minimum week: three DSA problems, two hours on the capstone and a small number of high-fit job-search actions.

Roadmap 1 | Backend + senior engineering foundations

Goal: become interview-ready for Java backend roles while strengthening architecture judgment, maintainability, reliability and production debugging. Python is a second tool, not a reason to abandon Java.

Phase	Focus	Must-learn topics	Practical output
1 / Weeks 1-2	Java + Spring refresh	Modern Java relevant to roles; Spring Boot REST; validation; error handling; config; dependency injection; JUnit/Mockito; Git; Maven/Gradle.	Spring Boot API with PostgreSQL persistence, tests and clear README.
2 / Weeks 2-4	SQL, security and API quality	Joins/aggregates; schema/indexes; transactions/isolation; pagination; auth; migrations; API contracts; test strategy.	Explain data model and transaction boundaries; test failure cases.
3 / Weeks 3-6	Concurrency + JVM	Executors; CompletableFuture; locks/atomics; happens-before; thread safety; virtual threads; GC basics; thread dumps; JFR/JMC.	Reproduce a race, fix it, measure cautiously and write a diagnostic note.
4 / Weeks 4-8	System design	Requirements/SLOs; capacity estimates; caching; queues; consistency; retries/timeouts; idempotency; backpressure; resilience; observability.	Practice 4-6 design interviews; write one 2-page design doc.
5 / Weeks 7-10	Kafka + distributed workflows	Partitions/keys; order; consumer groups/offsets; retries/DLQ; schema evolution; idempotent consumers; transactional outbox.	Move one workflow to events; demonstrate retry and duplicate handling.
6 / Weeks 8-12	Containers + delivery + cloud	Docker/Compose; GitHub Actions; secrets/IAM; deployment; health checks; logs/metrics/traces; rollback; cost awareness.	Automated build/test pipeline and a deployed or reproducibly runnable service.

Interview practice throughout

- DSA: arrays/strings, hash maps, stacks/queues, linked structures, trees, graphs, binary search, sorting, recursion, complexity and common patterns.
- Backend: SQL, transactions, REST/API design, tests, security, concurrency, error handling, production incidents and practical trade-offs.
- System design: clarify requirements, estimate load, draw components and data flow, identify failure modes, then defend trade-offs. Avoid microservices/Kubernetes/caching without a reason.

Roadmap 1 | Resources, senior habits and project criteria

Choose one main resource per topic and use a second only to resolve a gap. The practical output matters more than course completion.

Resource shortlist

Pluralsight - Java concurrency and multi-threading - Selected sections for concurrency and common patterns.

Pluralsight - Java Performance Tuning - JVM performance concepts after the concurrency basics.

Udemy - Java Multithreading, Concurrency & Performance Optimization - Use only if available through your BCCLS access.

Pluralsight - Docker for Software Development - Start with Docker Foundations, Compose and one app-containerization course.

Pluralsight - Kubernetes for Software Development - Optional after Docker; learn application deployment, configs, probes, logs and rollouts.

System design video support - Gaurav Sen for concepts; ByteByteGo for visual reinforcement; always practice the design yourself.

Designing Data-Intensive Applications - Use selectively for transactions, replication, partitioning, consistency and stream processing; check library access.

Senior-level habits to build in every phase

- Design ownership: document constraints, alternatives, consequences and reversal conditions in short ADRs.
- Reliability: reason about timeout/retry budgets, idempotency, consistency, graceful degradation, recovery and alerts.
- Quality: unit/integration/contract tests, cohesive modules, safe migrations, API contracts, deployment and rollback docs.
- Operations: measure before optimizing, correlate logs/traces, create a runbook and write a small incident analysis.
- Senior communication: explain trade-offs, clarify requirements, review code constructively and help others understand the system.

Optional: low-latency Java

The two low-latency PDFs are specialist extensions, not your starting point. First learn concurrency, JVM diagnostics and measurement here. Later, if job descriptions require it, study JMH, JFR, async-profiler, Java NIO, Linux scheduling and CPU/cache behavior. NUMA and vendor-specific networking come last.

Roadmap 1 | Portfolio project and readiness checks

Start with a project small enough to finish while job hunting. Do not build the entire future architecture before the first release.

Project v1: Document Processing API

- Spring Boot REST API accepts a document or record, validates input, stores metadata in PostgreSQL and returns structured errors.
- Include auth/authorization appropriate to the scope, tests, migrations, request IDs, logs and API documentation.
- README describes the problem, architecture, local setup, tests, example requests, trade-offs, limitations and next steps.

Upgrade it as you learn

Upgrade	Add	What it proves
Reliability	Idempotency keys, bounded retries/timeouts, rate limiting, structured errors.	You design for failure and repeated requests, not just the happy path.
Concurrency	Asynchronous processing, bounded queue/executor, safe shared state and a load test.	You understand concurrency correctness and resource limits.
Events	Kafka + outbox + idempotent consumer + retry/DLQ.	You can explain asynchronous workflows and consistency boundaries.
Delivery	Docker Compose, CI/CD, health/readiness checks and one cloud deploy.	Another engineer can run, test and deploy it.

You are ready to lead with this project when you can:

- Walk through the architecture and explain why each major component exists.
- Explain schema/indexes, transaction boundaries, security, tests and failure behavior.
- Describe one bug or performance issue, the evidence you gathered and the fix you validated.
- Run it from documented steps and show tests passing. Reproducibility is better than a fragile demo.

A project is evidence, not a substitute for applying

Apply from week 1. From week 8 onward, use the questions that recur in interviews to select the next study topic.

Roadmap 2 | Data engineering + DevOps/platform

Goal: build and operate dependable data pipelines and the platform around them. Data engineering moves and validates reliable data; data science analyzes data and develops statistical or predictive models. They overlap, but they are not the same job.

Prerequisites

Basic Java/Python, SQL, Git, REST, and the first Docker exercises from Roadmap 1. Do not wait until you have mastered Kubernetes or AI agents.

Phase	Focus	Topics	Checkpoint
1 / Weeks 1-2	SQL + data modeling	Joins/window functions; normalization vs dimensional modeling; fact/dim tables; grain; keys; data contracts; OLTP vs OLAP.	Design source and analytics schemas; explain data grain and metric definitions.
2 / Weeks 2-4	Python pipelines	JSON/CSV/Parquet; pandas where useful; schemas; config/secrets; logging; retries; tests and validation.	A repeatable ingestion job that validates records and writes clean output.
3 / Weeks 4-6	Transforms + orchestration	ETL vs ELT; dbt models/sources/tests/docs; schedules, dependencies, retries, backfills and idempotency.	A scheduled pipeline with data tests and documented recovery/backfill steps.
4 / Weeks 6-9	Batch + streaming	Kafka; event schemas; duplicate/late data; offsets; ordering; batch vs streaming trade-offs.	Add an event-fed pipeline and compare it with a batch path.
5 / Weeks 8-12	Cloud data services	One cloud: object storage/data lake, catalog, query engine, managed ETL, warehouse, IAM, secrets, logs and cost.	Small cloud pipeline with least-privilege access and observable runs.
6 / Weeks 10-16	DevOps/platform	Docker; GitHub Actions; Terraform basics; environment configs; deploy; alerts/monitoring; rollback; backups/runbooks.	CI/CD and runbook for a pipeline; failure alerts and clear recovery path.

Scope control

Begin with SQL + modeling + one transformation/orchestration approach before adding Spark. Learn Kubernetes only when target roles call for it; Docker, CI/CD, cloud IAM, monitoring and recovery come first.

Roadmap 2 | Resources and portfolio

Learn one tool in each category first. Official documentation is useful for tools whose APIs change; use a course for structure and docs for detail.

dbt Fundamentals - official training - About 5 hours; modeling, sources, tests, docs and deployment.

Apache Airflow documentation - Workflow-as-code, schedules, dependencies, retries and backfills.

AWS Skill Builder - Core cloud concepts; focus on IAM, S3, Glue, Athena and one warehouse option.

AWS data lakes and analytics overview - Service map; learn via one pipeline, not service memorization.

GitHub Actions quickstart - Automate tests, build and deployment.

Terraform AWS tutorials - Infrastructure as code after understanding the resources.

Pluralsight Docker for Software Development - Pick foundations, Compose and one language-specific course.

Pluralsight Kubernetes for Software Development - Optional developer-focused Kubernetes path.

Alternate if target jobs are GCP/BigQuery-heavy - Coursera Google Cloud Data Engineering certificate; use this instead of, not in addition to, the AWS cloud path.

Portfolio: Data Pipeline for Document/Event Analytics

Flow: API/Kafka events -> raw object storage -> validation/cleaning -> SQL/dbt transformations -> query/warehouse layer -> data quality and freshness checks -> schedule/alerting. Start locally and add cloud resources after the design works.

Acceptance criteria

- Reruns do not silently duplicate/corrupt data; pipeline tasks are idempotent.
- Bad records are quarantined or reported, and quality tests fail visibly.
- A backfill can be run for a date/range; late or duplicated records have a clear policy.
- README has flow diagram, schemas, sample queries, run status/alerts, recovery steps and cost assumptions.

Roadmap 3 | AI engineering + ML/data-science foundations

These are related but distinct directions. AI engineering is the nearer extension of backend work: reliable APIs that use LLMs, retrieval, tools and evaluation. Data science/ML requires a deliberate statistics/modeling foundation. Start with the AI-backend path; decide later whether you want deeper model work.

AI ENGINEERING	LLM APIs, structured output, RAG, evaluation, security, latency/cost, tool use and production operations.
DATA SCIENCE / ML	Exploratory analysis, probability/statistics, model validation, feature engineering, classical ML, then deep learning/model operations if needed.

AI engineering sequence

Phase	Learn	Checkpoint
1 / Python bridge	Python idioms/type hints; environment/package management; pytest; HTTP; async/await; FastAPI; basic pandas/NumPy only as needed.	A tested Python API called by the Java service.
2 / LLM fundamentals	Tokens/context; prompting; structured output; tool/function calling; streaming; retries/timeouts; model choice; latency/cost and data safety.	A bounded LLM API with good errors, logs and usage tracking.
3 / RAG	Document parsing/chunking; embeddings; vector similarity; pgvector; keyword + semantic retrieval; source citations; access control and prompt-injection risks.	Document Q&A; that cites sources and uses a fixed evaluation set.
4 / Evaluation + operations	Retrieval/answer quality; test sets; failure analysis; latency/cost monitoring; traces; model/prompt versions; rollback.	An evaluation report showing errors and measurable trade-offs.
5 / Tool workflows	Narrow tool schemas; permissions; human approval; workflow state; retries/timeouts; audit trail.	One useful multi-step workflow after RAG quality is measurable.

Do not start with agents or fine-tuning

First ship a reliable API, then basic RAG, then evaluation. Multi-agent frameworks, fine-tuning and custom model serving are follow-on specializations, not prerequisites for useful AI-enabled backend work.

Roadmap 3 | Resources and AI project

Use short courses for concepts, official documentation for implementation details, and a single evaluation-driven project for depth.

DeepLearning.AI course catalog and short courses - Start with ChatGPT Prompt Engineering for Developers, Building Systems with the ChatGPT API, LangChain Chat with Your Data and Building/Evaluating Advanced RAG.

Coursera - Generative AI with Large Language Models - Use the course/specialization referenced in the supplied roadmap; find the exact title in Coursera Plus.

FastAPI official tutorial - Reference while building the Python service.

Hugging Face learning hub - Later: transformers, models and datasets. Do not start with fine-tuning.

fast.ai Practical Deep Learning - Practical model-building for the ML lane after Python/data basics.

Scikit-learn getting started - Classical ML, pipelines, validation and metrics.

Portfolio: Document Intelligence / Knowledge Service

- Java/Spring Boot owns the business API and core persistence; Python/FastAPI owns document ingestion, embeddings and AI orchestration.
- Store metadata and access permissions in PostgreSQL; use pgvector if appropriate. Retrieval must respect document permissions.
- Return grounded answers with source references. Record model/prompt versions, retrieval results, latency, failure type and cost where available.
- Build a small evaluation set of representative questions and expected sources. Measure retrieval and answer quality; do not rely on impressive demo examples.
- Test prompt injection, missing info, oversized documents, rate limits, timeouts and provider outages.

Ready to show when

- You can explain why RAG fits the problem and its failure modes, compared with fine-tuning.
- You can show how a change to chunking/retrieval/prompting affects the same test set.
- You can describe production controls: security, privacy, cost/latency limits, monitoring and rollback.

Roadmap 3 | Data science/ML and the supporting math

Learning LLM APIs does not automatically qualify you for data science or ML-engineering roles. If those roles are a real target, use this branch intentionally.

Step	Topics	Evidence
1. Data analysis	Pandas/NumPy; missing data/outliers; distributions; exploratory data analysis; visualizations; leakage awareness.	Reproducible analysis report on a public dataset with assumptions/limitations.
2. Statistics	Conditional probability/Bayes; distributions; sampling; confidence intervals; hypothesis tests; A/B testing; correlation vs causation.	Choose and explain a statistical method; communicate uncertainty.
3. Classical ML	Train/test split; cross-validation; regression/classification; feature engineering; baselines; metrics; overfitting/regularization; error analysis.	Compare models with a baseline, check leakage and justify the metric.
4. ML engineering	Pipelines; reproducibility; experiment tracking; data/model versioning; serving; monitoring/drift; retraining triggers.	Deploy a simple model API and document monitoring decisions.
5. Deep learning - optional	Neural-network/backprop intuition; PyTorch/fast.ai; embeddings/transfer learning; fine-tuning only when justified.	A focused model experiment with evaluation, not just a copied notebook.

Math sequence

1. Algebra/functions/geometry/trigonometry -> 2. Discrete math for CS -> 3. Single-variable calculus -> 4. Linear algebra -> 5. Multivariable calculus -> 6. Probability/statistics -> 7. Optimization -> 8. ML-focused review.

For AI application engineering, prioritize embeddings/vector intuition, similarity, evaluation and uncertainty; calculus can wait. For data science/ML, statistics and linear algebra move up in priority. Some ML-math specializations include Python labs, but coding need not be mandatory for every math course.

Math resources

Khan Academy mathematics - Refresh gaps and English terminology; skip topics you can already solve reliably.

MIT OpenCourseWare - Mathematics for Computer Science - Logic, proofs, induction, counting, graphs, recurrences and probability.

Coursera - Mathematics for Machine Learning and Data Science - Structured linear algebra, calculus, probability and statistics for ML.

One integrated portfolio: three tracks, one system

The supplied plans repeatedly favor one serious production-style project over many unrelated tutorials. Upgrade the same system in deliberate stages so it demonstrates backend, platform/data and AI skills.

Java API Auth + validation	PostgreSQL Schema + transactions	Kafka Outbox + retries	Python API Ingest + RAG	Data layer Quality + analytics

Every component shares Docker/Compose, CI/CD, secrets/IAM, automated tests, logs/metrics/tracing, alerts, runbooks and a written architecture decision.

Build order

Milestone	Build	Evidence
1	Spring Boot + PostgreSQL + tests + README.	Roadmap 1 foundation.
2	Security, migrations, indexes, idempotency and useful logs.	Production-minded backend.
3	Docker Compose + CI pipeline + cloud deployment of one service.	Delivery/platform fundamentals.
4	Kafka workflow with outbox, retries and DLQ.	Distributed systems/data movement.
5	Python document ingestion + RAG with citations and an evaluation set.	AI engineering.
6	Scheduled/streaming data pipeline, transformations and quality checks.	Data engineering.
7	Optional model training, advanced performance or low-latency optimization based on target roles.	Chosen specialization.

Portfolio package

- README with local run/test steps, environment variables, API examples and limitations.
- Architecture diagram and short decision record on component boundaries and trade-offs.
- Tests and repeatable quality/latency measurements; do not make unsupported performance claims.
- Failure/recovery note showing what happened, evidence used, and how you would improve the system.

Choose emphasis by target job; do not learn everything

Target role	Lead with	Add next
Java backend	Java/Spring, SQL/PostgreSQL, testing, API design/security, DSA, concurrency, system design.	Kafka, Docker/cloud, observability; Python/AI as a differentiator.
Senior backend/platform	Architecture trade-offs, distributed systems, reliability, concurrency, production operations and communication.	IaC, SLOs, incident response and deeper profiling.
Data engineer	SQL, Python, data modeling, pipeline reliability, batch/stream, data quality, one cloud data stack.	dbt/Airflow/Spark/Kafka based on actual postings; warehouse/cost governance.
DevOps/platform	Linux/networking, Docker, CI/CD, cloud IAM/networking, Terraform, observability and recovery.	Kubernetes depth, GitOps, policy/security as required.
AI backend/RAG engineer	Backend API design, Python, LLM APIs, retrieval, evaluation, privacy/security, cost/latency and deployment.	Advanced RAG, bounded agents, LLMOps and model serving.
Data science / ML	Statistics, SQL/Python, EDA, modeling, validation and explaining uncertainty.	Deep learning, model pipelines, experiment tracking and production ML.

Rules for choosing courses

- Coursera Plus: use for a coherent specialization; Pluralsight: focused software/cloud paths; BCCLS Udemy access: selected hands-on courses; free official docs: implementation details.
- Check that a specific Udemy title is actually available through BCCLS before scheduling it. Avoid additional purchases unless there is a clear gap.
- Define the output before starting a course: feature, design note, deployment, pipeline, benchmark or evaluation report.
- Do not study AWS and GCP deeply at once; do not study Airflow, Dagster and Prefect at once; do not study multiple agent frameworks at once.
- Certificates are optional structure or a hiring signal; portfolio evidence and interview performance remain the objective.

Your recommended order

Now: Roadmap 1 + applications. Once the base service works: add delivery/cloud/Kafka. Then choose Roadmap 2 if data/platform roles are your priority or Roadmap 3 if AI-backend roles are your priority. Keep the other track for later; use the math sequence when moving toward ML/data science.

Optional extension | Low-latency Java and systems performance

The two supplied low-latency roadmaps overlap strongly. They are valuable for roles that explicitly mention sub-millisecond latency, trading/market data, Java NIO or hardware-aware performance, but they are not the starting point for a broad backend job search.

Order	Topic	Practical proof
1	Concurrency/memory model: races, visibility, atomics, locks, deadlocks, queues and false sharing.	Compare synchronized, AtomicLong and LongAdder; explain correctness and contention.
2	JVM/JIT/GC, allocation, warm-up and diagnostics.	Read GC logs/thread dumps or use a profiler to locate an evidence-backed issue.
3	JMH, JFR, Java Mission Control, async-profiler and p50/p95/p99 metrics.	Report before/after under controlled conditions; explain measurement limits.
4	TCP/UDP, Java NIO, Linux scheduling, backpressure and queueing.	Bounded producer-consumer pipeline with queue-depth/latency measurements.
5	CPU caches, branch prediction, NUMA, CPU affinity and vendor-specific networking.	Only if the target role actually requires this advanced depth.

Final operating principles

- The first objective is a job, not a perfect curriculum. Applications and interview practice start now.
- Strengthen fundamentals by explaining them and applying them in a project, not by collecting more courses.
- Keep Java as the strong backend base; use Python where it adds clear leverage for data/AI.
- Build senior skills through trade-off analysis, reliability, documentation, code review and ownership, not only advanced technology names.
- Choose specializations from role descriptions and interview feedback. Keep low latency optional until it is clearly relevant.

Resource access notes

Coursera Plus, Pluralsight and BCCLS Udemy availability can change by region, library arrangement or course plan. Confirm the exact course title is accessible before starting. Links in this roadmap point to official course catalogs or documentation where possible.